Safe Learning: a practical mathematical book

From rebuilding mathematical confidence to reasoning about learning systems that must respect constraints

> **BEGIN WITH A QUESTION YOU CAN ANSWER**
>
> If the notation feels unfamiliar, start with Chapter 0: mathematical language. If you can manipulate expressions but lose track of their meaning, use the chapter application labs: each begins with a concrete decision, introduces a model, works through the mathematics and interprets the result. The short readiness checks help locate a specific gap without turning the book into an entrance exam.

What this book teaches

A learning algorithm can optimize the quantity it was given and still make an unacceptable decision. A controller can perform well in a simulation and still fail when a sensor is delayed. A classifier can have high test accuracy without a useful guarantee near one particular input. The mathematics of safe learning starts by specifying the decision, the uncertainty and the precise claim to be established.

This book builds the tools needed to do that work. Six foundation chapters explain the language of proofs, linear algebra, optimization, probability, feedback systems and learning models. Fifteen later chapters develop constrained learning, safe exploration, control certificates and neural-network verification. Each chapter keeps a route through its essential ideas, worked calculations, interactive experiments, graded practice and more advanced material for a second reading.

The intended starting point is familiarity with elementary algebra, a first linear-algebra course and single-variable derivatives. You do not need to remember them fluently. The opening refreshers and exercises rebuild those skills. You also do not need a previous course in reinforcement learning, control theory or Gaussian processes: their definitions are developed before the later chapters use them.

There are two goals. First, become able to carry out a calculation and explain why each step is valid. Second, become able to ask whether the calculation answers the engineering question. The second skill requires identifying assumptions, choosing units, looking for counterexamples and recognizing when an apparently reassuring number is not the needed guarantee.

How to study a chapter

1. **Predict before calculating.** Read the scenario and say what increasing an error bound, tightening a constraint or changing a norm should do. A wrong prediction is useful: it gives the derivation something specific to resolve.
2. **Read with a small model beside you.** Write the variables and their units. Copy a worked example, leaving enough room to fill in every omitted multiplication or inequality. A matrix equation becomes less intimidating when its dimensions are visible.
3. **Use the chapter's two layers.** The main sections develop definitions, arguments and methods. The application lab near the exercises joins those ideas into a decision. Optional proof and literature boxes can wait until you can explain the core calculation.
4. **Attempt, hint, check, reconstruct.** Try an exercise before opening its hint. Use the solution to diagnose the first incorrect step, then close it and solve again. Copying a correct answer does not test whether you can recover the reasoning.
5. **Transfer the method.** Change a parameter, an uncertainty model or the actual requirement. Ask which line of the solution changes first. The harder application exercises are designed for this step.

One useful session is a small concept block, one worked example and two attempted exercises. When a topic takes several sessions, return to its definitions and a previously solved example before continuing. These are study suggestions, not a prescribed timetable. Let your ability to explain and reconstruct determine the pace.

The complete reading path

Read the parts in order for the fullest route. Within a focused track, follow every unfamiliar prerequisite link. Each application link below takes you directly to a chapter's practical synthesis; the chapter title opens its beginning.

Part I — Mathematical foundations

- 0. Mathematical Language, Proofs & Limits — application and review
- A. Linear Algebra II: Norms, Positive Definiteness & the SVD — application and review
- B. Calculus, Convexity & Optimization — application and review
- C. Probability, Concentration & Information — application and review

- D. Dynamical Systems, Stability & Feedback Control — application and review
- E. MDPs, Reinforcement Learning & Neural Networks — application and review

Part II — Safety and the mathematical toolkit

- 1. The Safe Learning Landscape — application and review
- 2. Math Toolkit I: Duality, LMIs & the S-Procedure — application and review
- 3. Math Toolkit II: Kernels, GPs & Uncertainty Bounds — application and review

Part III — Safe exploration

- 4. Safe Bayesian Optimization: SafeOpt & Controller Tuning — application and review
- 5. Is Safe BO Actually Safe? Real-β-SafeOpt and LoSBO — application and review
- 6. Global Safe Exploration of Dynamical Systems: GoSafe & GoSafeOpt — application and review
- 7. Viability & Safe Value Functions — application and review

Part IV — Constrained learning and control

- 8. CMDPs, Duality & Lagrangian Methods — application and review
- 9. Trust Regions, CPO & Modern Safe Policy Optimization — application and review
- 10. Barrier Functions, Reachability & Safety Filters — application and review
- 11. Lyapunov Certificates, Safe Model-Based RL & Learning-Based MPC — application and review

Part V — Certified neural networks and verification

- 12. Lipschitz Bounds via SDP: LipSDP and Beyond — application and review
- 13. Lipschitz-by-Design Networks & Direct Parameterizations — application and review
- 14. Neural Networks in the Loop: QCs, IQCs & Dissipativity — application and review
- 15. Verification & Distribution-Free Guarantees — application and review

Connected applications and projects

The examples use several recurring settings: noisy sensors, controlled temperature, robots near boundaries, constrained operating policies and learned decision scores. They are deliberately simplified teaching models with constructed numbers. Their value is that you can inspect the entire reasoning, including where the model stops being informative.

- A measured tank with a learned nominal controller combines interval uncertainty, a robust action filter, recursion and model failure. Start after Primers A–D and revisit after Modules 10–11.
- Tuning a controller without spending the safety margin combines bounded measurement error, Lipschitz certificates, safe exploration and a distinction between a safe query and a performance preference. Start after Modules 3–5.
- A learned decision under perturbation and distribution shift combines norm geometry, pointwise robustness, conformal calibration and a final argument about which guarantee answers which question. Start after Modules 12–15.

Each project includes a fully worked route and further exercises with hints and explained solutions. The projects do not ask you to operate a physical system. They ask you to build a model, make a decision within it and audit the resulting claim.

How to judge your progress

You are ready to move on when you can do four things: explain the main idea without reading its definition, solve a routine problem, solve a changed version of that problem, and identify an assumption whose removal breaks the conclusion. If only the algebra fails, return to a worked calculation. If the assumptions feel arbitrary, return to the counterexample or common-mistake discussion.

Stage	Evidence you can produce
Foundations	Compute a norm, derivative, conditional probability and one-step state update; explain their dimensions and meaning.
Modeling	Translate a verbal requirement into a constraint and describe the uncertainty that the constraint quantifies over.
Certification	Follow an invariant-set or optimization argument and locate the assumptions used at each critical step.

Stage	Evidence you can produce
Independent use	Complete a project, check its calculations, and explain both its guarantee and its limitations in ordinary language.

Keep a short error log: the mistaken step, why it failed, and a corrected example. Distinguish a notation error from a modeling error. Confusing an expectation with a worst-case bound is a different problem from dropping a minus sign, even when both lead to a wrong answer.

Conventions and evidence

The glossary connects recurring terms to examples and their full definitions. The formula sheet collects the reference mathematics; it is most useful after you understand a chapter's derivations. In a theorem, read the assumptions as part of the statement. "For all disturbances" and "with probability at least 0.95" are different quantifiers. A norm must be specified, a horizon must be identified, and a probability must refer to a stated source of randomness.

Worked engineering examples are hypothetical unless explicitly described otherwise. A bound proved within such a model is not empirical validation of that model. A simulation illustrates behavior for its chosen inputs; it does not establish a universal claim. Likewise, local Lean proofs check their encoded statements and assumptions, while the coverage records identify the remaining formalization gaps. The new application material has its own arithmetic and review records rather than inheriting a blanket verification claim.

To print a chapter, use your browser's print command. The print layout opens worked details and proof boxes so that a saved chapter includes the reasoning as well as the question. Interactive calculations remain available in the online chapter. Begin with Chapter 0, or choose your starting point with the study guide.